

Task-Aware Cloud-End Offloading for Vision-Language Model Serving via Dynamic Modality-Specific Adapter Scheduling

Zian Wang
Beijing University of Posts and
Telecommunications
Beijing, China
zianwang@bupt.edu.cn

Ziyi Wang*
Beijing University of Posts and
Telecommunications
Beijing, China
wangziyi0821@gmail.com

Jie Xing
Beijing University of Posts and
Telecommunications
Beijing, China
heiffeli@bupt.edu.cn

Yaya Wei
Omni-Channel Operation Center,
China Telecommunications
Corporation
Beijing, China
weiyi@chinatelecom.cn

Ziyan Zhong
Omni-Channel Operation Center,
China Telecommunications
Corporation
Beijing, China
zhongzy3@chinatelecom.cn

Lanshan Zhang*
Beijing University of Posts and
Telecommunications
Beijing, China
zls123@bupt.edu.cn

Abstract

Large-scale vision-language models enable powerful cross-modal understanding and generation, driving rapidly growing demand for online inference services. However, cloud-centric serving often suffers from high latency, rising costs, and network dependency, while purely on-device deployment is constrained by limited memory and reduced accuracy on complex tasks. To address this accuracy-latency-cost trilemma, we propose ShiftVL, a task-aware end-cloud serving framework that shifts suitable execution to the end device with a cloud fallback. ShiftVL serves high-frequency requests on an end-side small VLM enhanced with ViTexLoRA, a modality-disentangled parameter-efficient tuning method that preserves cross-modal alignment, while routing low-frequency or complex requests to a cloud-hosted large VLM for higher accuracy. Under tight device budgets, ShiftVL employs a predictive adapter scheduler that combines LRU-style caching with imitation learning to pre-load task-specific adapters. Experiments with InternVL models show that ShiftVL reduces cloud cost by up to 76.3% and latency by up to 42.9% while maintaining high multi-task accuracy, demonstrating its practicality for real-world vision-language model serving.

CCS Concepts

• **Computing methodologies** → **Cooperation and coordination; Planning for deterministic actions.**

Keywords

Vision-Language Model, Edge Computing, Parameter-Efficient Fine-Tuning, Predictive Scheduling

ACM Reference Format:

Zian Wang, Ziyi Wang, Jie Xing, Yaya Wei, Ziyan Zhong, and Lanshan Zhang. 2026. Task-Aware Cloud-End Offloading for Vision-Language Model Serving via Dynamic Modality-Specific Adapter Scheduling. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792127>

1 Introduction

Large-scale Vision-Language Models (VLMs) exhibit capabilities that go far beyond a mere combination of computer vision and natural language processing, enabling unified cross-modal understanding and generation [2, 19, 22]. They are versatile in handling a wide range of inference tasks, including recommendation, knowledge reasoning, information extraction, and image recognition etc. Leveraging these strengths, VLMs have already seen substantial adoption: the global multimodal AI market surpassed USD 2.5 billion in 2024 [8] and is projected to reach approximately USD 55.54 billion by 2035 [28], underscoring both their growing commercial value and the broad potential for future applications.

Despite VLMs are now widely deployed on commercial cloud platforms, their inference performance remains limited by two major factors: high operational costs for scaling GPU resources during peak demand, and network latency that can degrade real-time performance [6]. Deploying VLMs on end devices offers a promising solution, reducing reliance on cloud infrastructure and enabling inference under poor connectivity [5, 10, 26, 36]. However, limited memory and computational resources on end devices make it difficult to run models exceeding 8B+ parameters [29]. As a result, end-side VLMs remain unable to fully match the performance of cloud-based inference in practical scenarios.

The end-cloud collaborative inference scheme holds potential to resolve the VLM inference trilemma: the challenge of simultaneously optimizing accuracy, latency, and cloud computing cost. Its core idea is to leverage the advantages of both cloud and end for inference: offloading some simple tasks from the cloud to user terminal devices, thereby alleviating the cloud load [7]. However, existing collaboration methods are limited by substantial communication

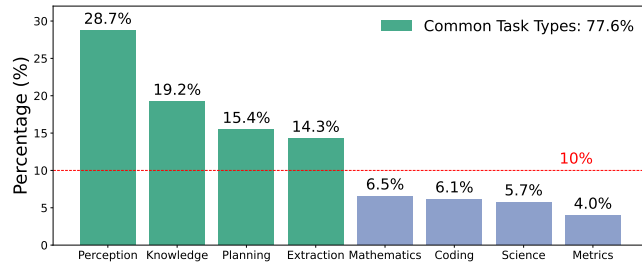
*Corresponding Authors: Ziyi Wang and Lanshan Zhang.



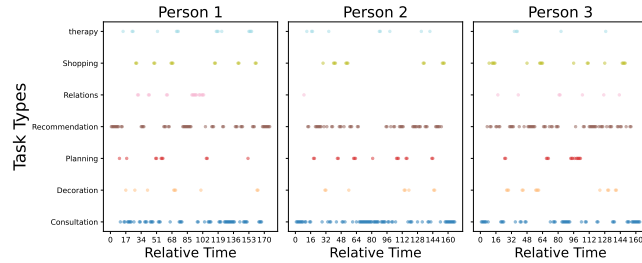
This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792127>

overhead arising from frequent end-cloud interactions [15]. Current end-cloud collaboration schemes can be broadly categorized into two types: The first is single-model partitioning [17, 20], which involves splitting a single model into multiple sub-computation graphs deployed on the end and cloud, respectively. This method requires frequent transmission of intermediate features, resulting in significant communication overhead, notably increased inference latency, and susceptibility to network fluctuations. The second is multi-model collaboration [12, 16, 34], a typical example being speculative decoding. It is a token-wise collaborative mechanism: the end side runs a lightweight model for preliminary inference, which is then corrected or supplemented by the large cloud model. However, the effectiveness of this approach heavily relies on the error correction capability of the large cloud model and also requires high-frequency end-cloud communication (see Section 2).



(a) Common Task Types Account for the Majority of User Requests.



(b) Temporal Distribution of Request Types Across Different Users.

Figure 1: Users’ vision-language inference task flows. Subfigure (a) presents real-world task types and proportions from MEGABench [1]; Subfigure (b) presents long-term interactions between users and chatbots from PersonaMem [14].

To address the VLM inference service trilemma, we analyze real-world vision-language request traces and observe that user demand is highly structured rather than uniformly distributed. Figure 1a summarizes the overall task composition. It shows a clear workload skew where a small set of common task types contributes the majority of requests. In particular, high-frequency categories such as perception-oriented queries and extraction-style requests together account for over 75% of the total volume. This indicates a pronounced long-tail pattern in practice. As a result, optimizing these frequent categories on the end side can substantially increase the fraction of requests served locally and reduce reliance on cloud execution. Beyond the aggregate distribution, the request stream also exhibits temporal structure. Figure 1b visualizes the task-type

sequences of several representative users over time. While different users show different preferences, each user’s requests tend to recur within certain time windows, forming regularities that can be exploited. This motivates proactive end-side preparation. With multiple task-specialized experts available on the end, the system can pre-load the most relevant expert modules or prefetch memory-augmented context based on recent behavior, thereby reducing task-triggered loading latency.

Building on these trace-driven findings, we propose ShiftVL, a task-aware cloud-end offloading framework for vision-language model serving. ShiftVL departs from cloud-centric serving by making request-wise execution decisions, shifting suitable requests to the end while retaining a cloud fallback path. Leveraging skewed workloads, ShiftVL serves high-frequency tasks on the end using a small VLM augmented with task-specific parameter-efficient adapters (implemented with LoRA-style modules [11]), improving end-side accuracy while reducing latency and cloud usage. Low-frequency or complex requests are handled by a cloud-hosted large VLM for higher accuracy. Under end-side constraints, ShiftVL leverages recurring task patterns to predictively stage task-specific adapters. ShiftVL includes two key components: a modality-aware tuning method (ViTexLoRA, Section 3.2) and the end-side adapter scheduler (Section 3.3).

The cross-modal reasoning of VLMs depends on accurate visual-text alignment. However, existing parameter-efficient fine-tuning methods (e.g., LoRA and its variants) are largely designed for uni-modal adaptation and often fail to capture multimodal interactions. We therefore propose ViTexLoRA, a fine-tuning method that uses modality-decoupled LoRA adapters for the visual and textual pathways to reduce interference while preserving intra-modal representations. ViTexLoRA further introduces a learnable channel-weighting mechanism to maintain cross-modal semantic alignment during tuning.

End devices have limited memory and storage, making it infeasible to keep many adapters resident. To reduce cold starts in a dynamic serving setting, ShiftVL uses a predictive adapter scheduler that combines LRU-style caching with imitation learning, motivated by the temporal regularities in Figure 1b. LRU absorbs short-term bursty requests, while imitation learning captures longer-term user access patterns. The scheduler models request history, task switches, and end-side resource status to predict the next task and pre-stage its adapters, reducing loading and transfer overhead.

We implemented the end and cloud prototypes of ShiftVL on NVIDIA RTX 2060 Super and A100 GPUs, respectively, connected via wireless network. The InternVL-2B and InternVL-9B models [2] were deployed as the SVLM and large LVLM, respectively. Performance evaluation was conducted using simulated user datasets derived from publicly available benchmarks. Experimental results demonstrate that ShiftVL reduces cloud computing resource usage by up to 76.32% and task processing latency by up to 42.94%, while maintaining relatively high multi-task accuracy.

- ShiftVL is a task-aware cloud-end collaboration framework for VLM serving, combining an end-side fast path for frequent tasks with a cloud fallback for long-tail requests to reduce cloud cost and end-to-end latency while preserving multi-task accuracy.

- ViTexLoRA is a VLM-specific fine-tuning method that disentangles visual/text adaptation and maintains cross-modal alignment, improving end-side accuracy on high-frequency tasks.
- We design a predictive parameter-efficient adapter scheduler that pairs LRU-style caching with imitation learning to stage adapters ahead of time, increasing local serving coverage and reducing cold starts.

2 Background and Motivation

This section explores two key aspects: the limitations of existing collaboration schemes and the evolution of token attention across layers in vision-language models, which provide insights into designing more effective collaboration frameworks and fine-tuning methods.

2.1 Limitations of Current Schemes

We focus more on the research path of multi-model collaborative approaches due to the high communication latency, unavailability under weak network conditions, and limited flexibility of dynamic adjustment of single-model partitioning approaches. In contrast, the mainstream solution of multi-model collaborative approaches typically adopts the speculative decoding mechanism, where an SVLM drafts and a cloud LVLM co-verifies in a token-wise manner. Its theoretical basis lies in the empirical observation that the output probability distribution of SVLM in simple token generation tasks is often similar to that of LVLM, which makes partial substitution feasible.

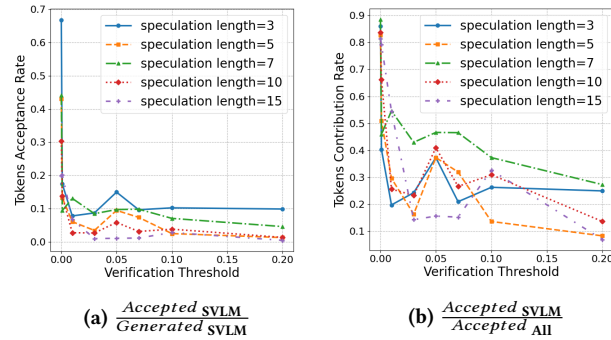


Figure 2: Impact of verification threshold and speculation length on speculative decoding. Subfigure (a) presents the acceptance rate of SVLM-generated tokens; Subfigure (b) presents the SVLM token contribution ratio in the output.

However, this mechanism suffers from several inherent limitations. First, token-wise collaboration often results in the small model contributing marginally to the final output. As shown in Figure 2a, across different parameter settings, the acceptance rate of drafts generated by the small model is generally low and further declines as the verification threshold rises. This means that the large model rarely adopts the small model’s draft content, diminishing the purpose of collaboration. Moreover, Figure 2b shows that the proportion of tokens contributed by the small model remains consistently low, often below 0.4.

Second, speculative decoding does not fundamentally alleviate the reliance on cloud resources. While intuitively one might expect the small model to reduce cloud workload, in practice, the verification process introduces significant overhead. Our tests show that for a generation length of 300 tokens, an average of 182.05 additional verifications are required, leading to about a 60% increase in cloud computing costs.

In essence, token-wise collaboration struggles to reconcile the trade-off between accuracy, latency, and cost. The small model’s contribution is too limited to offset the verification overhead, while the cloud still bears most of the computational burden. These limitations motivate us to rethink the granularity of collaboration.

In contrast, we propose a request-wise model collaboration approach. By shifting from token-wise to request-wise allocation, communication overhead is substantially reduced and end-to-end latency is effectively lowered. More importantly, the task allocation strategy allows us to preserve inference accuracy while ensuring that cloud resources are only invoked when necessary, striking a more balanced compromise among latency, cost, and accuracy. Details of this approach will be presented in the next chapter.

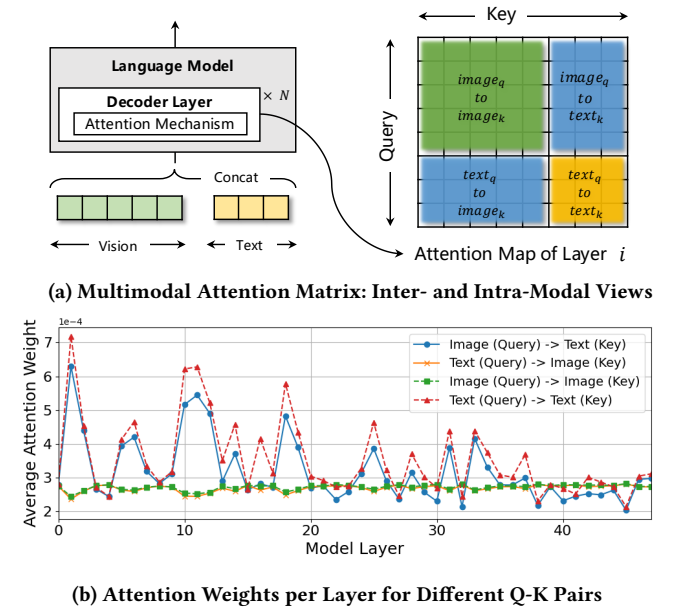


Figure 3: Exploring bimodal attention mechanism in vision language models. Subfigure (a) illustrates intra- and inter-modal attention between image and text; Subfigure (b) shows average attention weight distributions across layers for different query-key pairs.

2.2 Token Importance in VLMs

We analyze the correlation between tokens in InternVL-9B’s attention maps to observe their modality fusion patterns. We divide each attention layer’s full matrix into four sub-regions— intra-modal and inter-modal (as shown in Figure 3a). We then calculate the average attention weight within each sub-region. Figure 3b illustrates the attention evolution pattern, revealing that in the model’s shallow

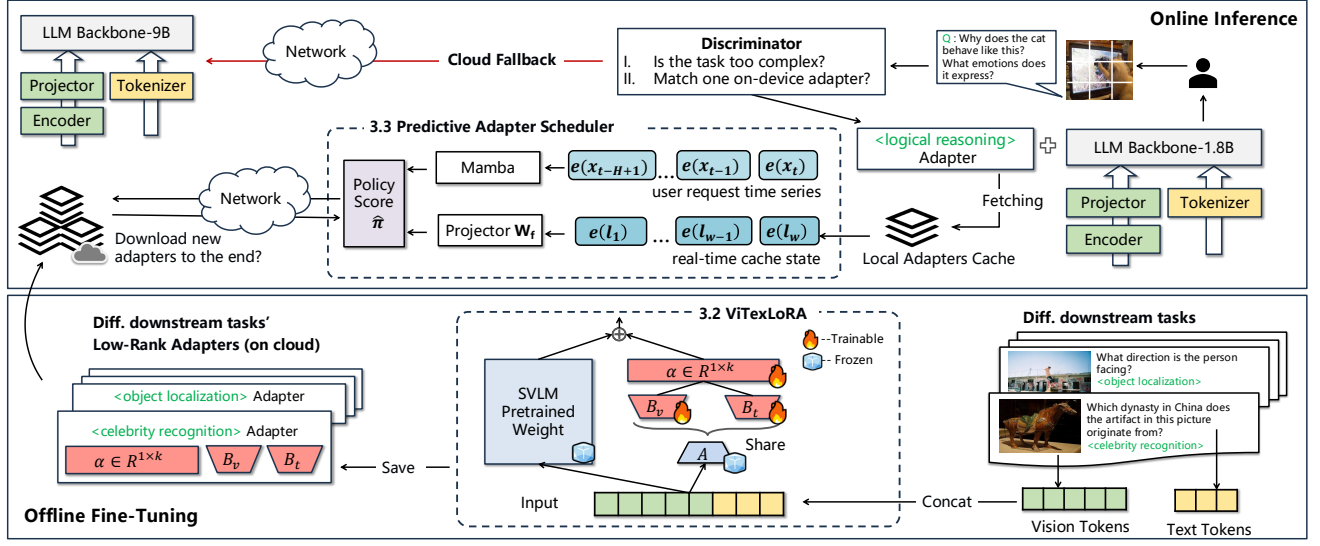


Figure 4: Framework of ShiftVL. Online serving (top): the Discriminator selects the end-side fast path or the cloud fallback, and a predictive adapter scheduler prepares task-specific adapters on the end; Offline tuning (bottom): ViTeXLoRA trains task-specific adapters in the cloud.

layers (e.g., Layers 0–8), attention resources are primarily concentrated on textual information. This suggests that deep bidirectional inter-modal fusion does not occur in the early stages.

In particular, we observe that $text_q \rightarrow text_k$ and $image_q \rightarrow text_k$ attention weights are both dominant and highly fluctuating in the shallow layers. This indicates that the model heavily relies on the textual modality for context understanding and semantic initialization during its initial phase, while image queries actively seek textual grounding for disambiguation. Such behavior reflects an implicit asymmetry in which text functions as the semantic anchor whereas vision initially plays a complementary role.

As the layer depth increases, $text_q \rightarrow text_k$ and $image_q \rightarrow text_k$ attention show a downward trend. Rather than a mere decline in importance, this shift suggests that the model has already extracted key semantic cues and re-encoded them into abstract joint representations. In parallel, intra-modal vision attention ($image_q \rightarrow image_k$) gradually stabilizes, implying that visual features increasingly rely on their own structure once textual guidance is absorbed. The persistent asymmetry between modalities indicates that fusion is not uniform but dynamically reweighted across layers, with different roles of “guidance” and “autonomy” emerging at different depths.

These observations highlight that modality-specific preferences evolve hierarchically, yet current parameter-efficient fine-tuning strategies, such as LoRA apply homogeneous adjustments across all layers. Such uniform treatment fails to capture the heterogeneous fusion dynamics, particularly when one modality requires strong anchoring in the early stages while the other demands greater autonomy at deeper levels. To address this limitation, we propose a modality-decoupled fine-tuning strategy that explicitly accounts for the distinct learning trajectories of text and vision across different layers, thereby enabling more precise alignment and more efficient adaptation in large-scale VLMs.

3 Design

3.1 Overview of ShiftVL

Figure 4 overviews ShiftVL, which couples offline adapter training in the cloud with online cloud-end serving to enable task-aware cloud-end offloading for vision-language requests.

ShiftVL uses ViTeXLoRA to enhance the end-deployed SVLM with task-specific parameter-efficient adapters. ViTeXLoRA decouples adaptation in the visual and textual pathways with modality-specific low-rank parameters while retaining a lightweight shared component to preserve cross-modal alignment, reducing interference during multimodal fusion. Task-specific adapters (e.g., object localization, celebrity recognition, logical reasoning) are trained in the cloud and periodically synchronized to the end, allowing the SVLM to serve common requests locally while the cloud-hosted LVLM handles complex or infrequent queries. For each request, ShiftVL performs task semantic categorization and selects either the end-side fast path or the cloud fallback. A lightweight classifier combines a Transformer encoder with unsupervised clustering to map multimodal embeddings into latent task categories; if the required adapter is available and the SVLM is expected to meet accuracy targets, inference runs on the end, otherwise it falls back to the LVLM. Under tight end-side storage budgets, ShiftVL uses a predictive adapter scheduler that combines LRU-style caching with imitation learning to anticipate upcoming tasks and stage the corresponding adapters, reducing cold-start latency and transmission overhead.

3.2 ViTeXLoRA

The basic LoRA assumes that the weight updates during fine-tuning reside in an ‘intrinsic low-rank subspace’. Instead of updating the full pre-trained weight matrix, LoRA represents the update

as the product of two smaller matrices, making fine-tuning more parameter-efficient. Specifically, the weight updating of LoRA can be expressed as:

$$W' = W_0 + \Delta W = W_0 + BA \quad (1)$$

where $W_0 \in \mathbb{R}^{d \times k}$ represents the original pre-trained weight, $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank decomposition matrices, and r is a rank hyperparameter that satisfies $r \ll \min(d, k)$.

When applied to vision-language models, conventional LoRA variants typically employ shared parameters for all modalities, treating visual and textual tokens uniformly. However, such uniform fine-tuning may limit each modality's ability to fully exploit its unique characteristics. Analyses of InternVL-9B attention patterns (Section 2.2) suggest that text and vision play different roles at different layers, with text often acting as the semantic anchor in early layers while visual features gradually gain autonomy in deeper layers. These observations indicate that an effective fine-tuning approach should consider each modality independently, capturing and preserving modality-specific information throughout forward propagation.

Considering this, we propose ViTexLoRA, which consists of three components: a shared matrix A , a dual-modal matrix $B = [B_v; B_t]$, and a channel-wise reweighting vector α . The shared matrix A compresses input features into a unified latent space, while B_v and B_t capture modality-specific patterns for visual and textual inputs, respectively. The reweighting vector α enhances model expressiveness by adaptively scaling output channels. The parameter update in ViTexLoRA is formulated as:

$$\begin{aligned} W'x &= W_0x + \Delta W(P_vx + P_tx) \\ &= W_0x + \underline{\alpha} \odot (\underline{B_v}AP_vx + \underline{B_t}AP_tx) \end{aligned} \quad (2)$$

where the underlined parameters indicate the trainable components. P_v and P_t are diagonal masks that distinguish between different modalities. They satisfy the relation $P_v + P_t = I$, where I is the identity matrix.

During inference, the shared matrix A remains fixed and reusable across tasks, so the local device only stores it once. When adapting to a new task, only the task-specific matrices B_v , B_t , and the reweighting vector α are transmitted from the cloud. This design significantly reduces communication overhead and memory consumption, enabling resource-constrained edge devices to support multiple adapters efficiently.

3.3 Predictive Adapter Scheduler

ShiftVL dynamically updates LoRA adapters on the end-side device based on user access patterns to optimize inference under limited memory. While the Belady algorithm is theoretically optimal for cache replacement by evicting items with the longest future reuse distance [31], it requires full knowledge of future accesses and is therefore only applicable in an ideal offline setting. In real systems, access sequences are observed in a forward-only manner, making Belady impractical. To address this, the scheduler leverages imitation learning (IL) to capture Belady's long-term replacement planning, while delegating short-term decisions to a classical LRU mechanism.

Algorithm 1 Adapter Scheduling via Imitation Learning and LRU

Require: Total cache size w , LRU-controlled cache size v , sliding window length H , user access sequence $X = (x_1, x_2, \dots, x_n)$

Ensure: Updated caches $\mathcal{L}_{IL}$ and $\mathcal{L}_{LRU}$

```

1: Initialize IL-controlled cache  $\mathcal{L}_{IL}$  with  $w - v$  slots
2: Initialize LRU-controlled cache  $\mathcal{L}_{LRU}$  with  $v$  slots
3: for  $t = 1$  to  $n$  do
4:    $x_t \leftarrow$  current user request
5:   Form state vector  $s_t = (X_{t-H+1:t}, \mathcal{L}_{IL} \cup \mathcal{L}_{LRU})$ 
6:    $\mathcal{Y} \leftarrow$  IL_Policy( $s_t$ )
7:   if  $\pi_i = 1$  in  $\mathcal{Y}$  then
8:     Replace the slot  $i$  in  $\mathcal{L}_{IL}$  corresponding to  $\pi_i = 1$ 
9:   else if  $\pi_{lr_u} = 1$  in  $\mathcal{Y}$  then
10:    Replace an entry in  $\mathcal{L}_{LRU}$  using standard LRU policy
11:   else if  $\pi_{skip} = 1$  in  $\mathcal{Y}$  then
12:    Do nothing, cache remains unchanged
13:   end if
14:   Update access timestamps in  $\mathcal{L}_{LRU}$  for LRU tracking
15: end for
```

Problem formulation. The total cache of size w is divided into two regions. The first region contains v entries managed by LRU, providing a simple yet effective fallback for frequently accessed adapters. The remaining $w - v$ entries are managed by an imitation learning policy, which is trained to predict the optimal replacement actions within its controlled space. Let $X = (x_1, x_2, \dots, x_n)$ denote the sequence of user cache accesses, where x_i represents the task category requested at time step i . The end-side memory state is represented as $\mathcal{L} = (l_1, l_2, \dots, l_w)$. Given the access history X and the current request, the system forms the temporal sequence $X_t = (x_{t-H+1}, \dots, x_t)$, where H is a hyperparameter representing the window length, and the corresponding memory state $\mathcal{L}_t$, resulting in a state vector $s_t = (X_t, \mathcal{L}_t)$. Based on s_t , the imitation learning component of the scheduler generates a decision vector:

$$\mathcal{Y} = (\pi_1, \pi_2, \dots, \pi_{w-v}, \pi_{lr_u}, \pi_{skip} \mid s_t) \quad (3)$$

where π_i ($1 \leq i \leq w - v$) indicates whether to replace a specific slot in the IL-controlled region, π_{lr_u} delegates the replacement to the LRU-controlled cache, and π_{skip} indicates that no replacement is necessary. During inference, the system first obtains $\mathcal{Y}$ from the IL policy and then updates the cache according to the selected action (see Algorithm 1).

Decision generating. The imitation learning module of the scheduler receives as input the recent access window $X_{t-H+1:t}$ and the current cache state $\mathcal{L}_{IL} \cup \mathcal{L}_{LRU}$. The temporal dependencies in user access sequences are encoded via a state-space model (SSM)-based backbone, Mamba [9], which captures long-range sequential dynamics within the sliding window. These temporal features are then fused with the cache embeddings to produce a context-aware representation:

$$\begin{aligned} F_{out} &= \text{LayerNorm}(\text{Concat}(\mathcal{W}_f E(\mathcal{L}_t), \text{SSM}(X_t))) \\ \hat{\pi} &= \text{Softmax}(\text{MLP}(F_{out})) \in \mathbb{R}^{w-v+2} \end{aligned} \quad (4)$$

where $\mathcal{W}_f$ is a learnable projection matrix that aligns the cache and temporal feature spaces, while $E(\mathcal{L}_t)$ and $\text{SSM}(X_t)$ denote the embedded representations of cache state and user access sequence,

respectively. The internal formulation of $\text{SSM}(\cdot)$ follows the Mamba architecture and is detailed in the Appendix. The fused feature F_{out} is fed into the imitation-learning policy network, which outputs $w - v + 2$ logits $\hat{\pi}$ corresponding to different cache replacement actions.

Training strategy. During training, each step is labeled using Belady policies. The resulting supervision distinguishes three cases: (1) replacement within the IL-controlled space when Belady’s replacement target lies outside the LRU-managed protection region, (2) delegation to LRU when Belady’s replacement target falls inside the LRU-controlled region, and (3) skipping replacement when Belady does not select any cache row for replacement. The IL module is trained to output logits $\hat{\pi}$ corresponding to these decision types, with the binary cross-entropy loss defined as:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{w - v + 2} \sum_{i=1}^{w-v+2} [\pi_i \log \hat{\pi}_i + (1 - \pi_i) \log(1 - \hat{\pi}_i)] \quad (5)$$

At inference time, the IL policy first predicts the optimal action. If the decision is $\pi_{\text{IL}} = 1$, the replacement is handled entirely by LRU within its v -slot region; otherwise, the system follows the IL policy to either replace a slot in the $w - v$ region or skip replacement. This hybrid orchestration achieves near-optimal cache hit rates.

4 Experiments

In this section, we first evaluate the overall effectiveness of ShiftVL. Then, we analyze the impact of the framework’s two key modules through ablation studies. Finally, we examine the end-side overhead.

4.1 Experimental Settings

Datasets. We evaluate on three public multi-task VLM benchmarks: MMBench [24], SEED-Bench [18], and MME-RealWorld [40].

Baselines. We compare against representative deployment strategies: (1) Cloud-only, InternVL-9B [42]; (2) End-only, InternVL-2B [3]; (3) Token-wise cloud-end collaboration, SpecVLM [13] with draft verification every 10 tokens; (4) Model splitting, AlignedVQ [23], partitioning InternVL-9B 2:7 across end and cloud; and (5) Ablations, including ShiftVL-LoRA (replacing ViTexLoRA with standard LoRA) and ShiftVL-LRU (replacing our scheduler with LRU).

Metrics. We report end-to-end latency and its breakdown into Time to First Token (TTFT) and Time Per Output Token (TPOT), and include network overhead in the latency measurement. Cloud cost is estimated by token counting using the GPT-4o API pricing (\$2.50/1M input tokens, \$10.00/1M output tokens). Network cost is estimated using AWS data transfer pricing (\$0.09/GB). Detailed experimental settings are provided in the appendix.

4.2 Performance: Accuracy, Latency, and Cost

Figure 5a shows a clear accuracy ordering, with end-only performing worst and cloud-only together with the split-execution baseline performing the best. ShiftVL narrows this gap through hybrid cloud-end inference: frequent requests run on the end with a compact VLM enhanced by ViTexLoRA, while low-frequency or difficult requests fall back to the cloud-hosted large VLM. At the median, ShiftVL closes $\sim 59\%$ of the end-only to cloud-only gap and reaches $\sim 94\%$ of cloud-only accuracy. ViTexLoRA contributes consistently, as replacing it with standard LoRA shifts the accuracy CDF left.

ShiftVL-LRU is slightly more right-shifted in accuracy because more cache misses trigger cloud fallback, improving accuracy at the expense of latency and cost.

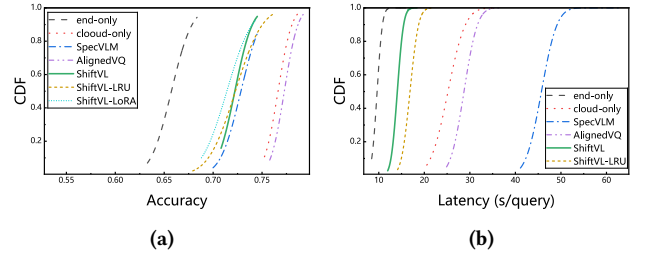


Figure 5: CDF of Request Accuracy and End-to-End Latency (higher accuracy and lower latency are better).

Figure 5b shows that, relative to cloud-only, ShiftVL reduces median latency by $\sim 42\%$, reaching high percentiles at smaller latency values. This gain comes from serving frequent tasks on the end fast path, where the small model is faster per token (InternVL-2B: 41.86 ms/token vs. InternVL-9B: 84.39 ms/token). SpecVLM has a much longer right tail, with median latency about $\sim 3\times$ that of ShiftVL, because speculative decoding introduces many cloud-end verification rounds and amplifies RTT overhead. AlignedVQ is also slower at high percentiles; split execution still transmits intermediate activations per token, leaving latency sensitive to RTT. Replacing the scheduler with pure LRU shifts the latency CDF rightward, consistent with more cold starts and fewer requests completing on the end fast path.

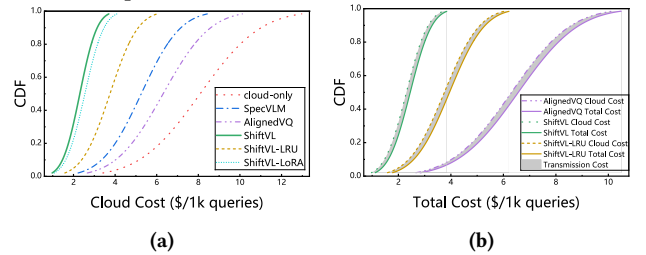


Figure 6: CDF of Cloud Cost and Total Cost per 1K Queries (lower is better); total cost includes cloud usage plus communication fees.

Figure 6a shows that ShiftVL has the most left-shifted and steepest cloud-cost CDF among cloud-end baselines, indicating lower and more predictable expenditure. Relative to cloud-only, ShiftVL cuts cloud cost by $\sim 76\%$ at both the median and P90 by shifting frequent tasks to the end and reserving the cloud mainly for long-tail complexity. SpecVLM and AlignedVQ remain more costly because cloud-side computation is substantial, due to repeated verification and the fact that the bulk of the workload still resides in the cloud. Compared with ShiftVL, ShiftVL-LRU increases median cloud cost by $\sim 35\%$, since more cache misses trigger cloud fallback. As shown in Figure 6b, we report total cost mainly for ShiftVL, ShiftVL-LRU, and AlignedVQ, since other baselines exchange only small text payloads and incur negligible communication overhead. ShiftVL and ShiftVL-LRU transmit adapters when needed, whereas AlignedVQ transfers intermediate activations with $\sim 96\%$ compression. Overall, adapter transmissions in ShiftVL are not frequent in practice, keeping its communication overhead acceptable.

Table 1: TTFT and TPOT under different RTT (mean $\pm$ std).

Method	TTFT (ms) $\downarrow$ better				TPOT (tokens/s) $\uparrow$ better			
	20	50	100	200	20	50	100	200
cloud-only	107.77 ± 9.29	138.17 ± 11.34	182.46 ± 8.91	283.53 ± 9.76	9.34 ± 0.76	7.28 ± 0.57	5.49 ± 0.26	3.53 ± 0.12
end-only	41.76 ± 3.79	42.10 ± 3.38	42.44 ± 4.06	42.42 ± 4.80	24.12 ± 1.91	23.89 ± 1.78	23.75 ± 1.96	23.81 ± 2.15
SpecVLM	526.36 ± 39.25	556.67 ± 36.68	606.84 ± 42.25	687.71 ± 48.64	3.51 ± 0.22	3.47 ± 0.19	2.84 ± 0.16	1.93 ± 0.08
AlignedVQ	97.65 ± 7.31	128.03 ± 8.95	173.66 ± 7.08	274.48 ± 7.65	10.30 ± 0.74	7.85 ± 0.53	5.77 ± 0.23	3.65 ± 0.10
ShiftVL	60.57 ± 3.84	69.48 ± 4.22	82.34 ± 4.01	111.13 ± 4.34	18.30 ± 1.22	18.19 ± 1.35	18.63 ± 1.30	18.50 ± 1.37
ShiftVL-LRU	92.00 ± 6.38	111.82 ± 7.71	140.62 ± 6.32	205.89 ± 6.72	15.92 ± 1.17	15.85 ± 1.37	16.43 ± 1.23	16.26 ± 1.28

4.3 Effect of Network Round-Trip Time

To quantify the impact of network round-trip time (RTT) on serving latency, we evaluate ShiftVL and representative baselines under four RTT settings (20/50/100/200 ms), covering low-latency LAN/MAN conditions and higher-latency wide-area or cross-region deployments. We report two complementary latency metrics in Table 1: TTFT for responsiveness and TPOT for sustained generation throughput.

Table 1 shows that ShiftVL consistently delivers low TTFT across all RTT regimes. At 20 ms RTT, ShiftVL reduces TTFT from 107.77 ms (cloud-only) to 60.57 ms, and it remains 2.5 $\times$ faster than cloud-only at 200 ms RTT (111.13 vs. 283.53 ms). Compared with cloud-end collaboration baselines, ShiftVL also maintains substantially lower TTFT, especially under higher RTT: at 200 ms, ShiftVL achieves 111.13 ms versus 274.48 ms for AlignedVQ and 687.71 ms for SpecVLM. This behavior matches the design of ShiftVL, where most frequent requests complete on the end fast path and avoid RTT-amplified interactions, while cloud fallback is used selectively.

For sustained generation, ShiftVL is markedly more robust to RTT variation. Its TPOT stays essentially unchanged when RTT increases from 20 ms to 200 ms (18.30 $\rightarrow$ 18.50 tokens/s, $\approx$ +1.1%), indicating that throughput is not bottlenecked by network delay in typical operation. In contrast, cloud-heavy baselines degrade sharply as RTT grows, with TPOT dropping by 62% for cloud-only (9.34 $\rightarrow$ 3.53) and 65% for AlignedVQ (10.30 $\rightarrow$ 3.65), while SpecVLM drops by 45% (3.51 $\rightarrow$ 1.93) due to repeated cloud-end verification. Overall, ShiftVL achieves strong responsiveness and stable generation speed across RTT regimes, demonstrating better robustness under adverse network latency.

4.4 Ablation Study of Key Modules

Fine-tuning module. We used various pre-training methods on the InternVL-2B base model, Lora, Dora [25], Adalora [39], and Hydralora [30], to compare with our ViTexLoRA approach across different datasets: MMBench, MME-RealWorld, and SEED-Bench. Due to overlapping task categories across different datasets, we separated and reaggregated samples based on specific task categories. For instance, the “Attribute” category includes subcategories such as “attribute_recognition” from MMBench, “instance_attributes” from SEED-Bench, and “vehicle_attribute” from MME. We reorganized the data into new datasets using this approach and divided each new category into training and evaluation sets at an 8:2 ratio for experimentation.

As shown in the Table2, the evaluation results demonstrate that ViTexLoRA achieves significant performance improvements on simpler tasks such as OCR. Therefore, we can leave such tasks to be processed by small models on the device without having to send requests to large models in the cloud. It should also be noted that due to the performance limitations of the 2B base model, even with fine-tuning using the peft method, it remains incapable of handling more challenging tasks such as “Count”. Therefore, we need to offload such tasks to the cloud, employing larger models to handle these categories of tasks. This reflects the intrinsic rationale and purpose of our ShiftVL: by distributing tasks of varying complexity between the end and the cloud.

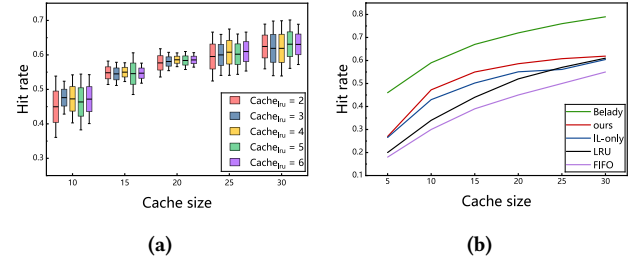


Figure 7: Hit-rate comparison of adapter scheduling policies on PersonaMem. Subfigure (a) shows Cache_{lru}’s impact; Subfigure (b) compares different methods.

Scheduling module. We evaluate the end-side adapter scheduling component on the PersonaMem dataset, which contains long-horizon user interaction traces. Figure 7a shows that, as the cache size increases, the hit-rate distributions shift upward consistently. By contrast, varying Cache_{lru} from 2 to 6 yields only modest changes and the boxplots largely overlap for each fixed cache size. This indicates that on PERSONAMEM traces, overall cache capacity is the primary driver of hit rate, while adjusting the LRU resident set size within this small range only mildly changes the eviction behavior. In practice, setting Cache_{lru} to a reasonable default is sufficient for stable performance, and further gains require signals beyond recency-only tuning.

Figure 7b compares our scheduler against FIFO, LRU, an imitation-learning baseline (IL-only), and the offline optimal upper bound (Belady). Our method consistently outperforms FIFO/LRU and IL-only across cache sizes, staying closer to Belady. The gain is most visible under tight caches; for example, at cache size 10, ours improves hit rate over LRU by about $\sim$ 14 percentage points, and the gap narrows as the cache grows and the curves converge. This behavior matches the intended “short-term + long-term” design: the LRU component absorbs bursty locality, while the predictor captures longer-range recurrence to reduce premature evictions. The gap is most evident at intermediate cache sizes; when the cache is extremely small, all policies are capacity-limited and many requests miss regardless of strategy, and when the cache is large enough to cover most frequently reused adapters, the benefit of smarter eviction naturally diminishes and the curves converge.

4.5 End-side Operational Overhead

We further evaluate the runtime overhead of the system on the end-side device, focusing on both latency and memory consumption.

Table 2: Evaluation results of LoRA, DoRA, AdaLoRA, HydraLoRA, and our ViTexLoRA on InternVL-2B. We categorized results by task type (left side) and data source (right side).

Methods	Attribute	Emotion	Count	OCR	Reasoning	Scene	Diagram	SEED-Bench	MME	MMBench	Avg.
InternVL-2B	68.75	59.66	42.27	84.54	58.14	72.42	50.82	67.09	40.79	61.89	58.61
InternVL-9B	74.53	69.93	50.00	98.68	74.66	85.27	67.83	72.82	52.55	79.93	73.01
LoRA	68.68	63.57	43.41	92.11	62.22	71.28	50.35	66.67	41.55	67.21	61.90
DoRA	70.01	62.84	42.50	89.47	57.92	72.43	49.18	66.53	41.06	67.10	61.71
AdaLoRA	68.95	57.70	43.41	93.42	58.14	71.28	50.35	66.43	41.46	66.37	61.33
HydraLoRA	70.55	60.88	42.27	88.16	63.80	72.92	50.82	67.46	41.23	66.43	61.52
ViTexLoRA	70.81	66.01	42.35	93.42	61.09	72.59	50.97	67.51	41.50	69.03	63.16

At the latency level, task classification (0.47–1.66 ms) and LoRA switching (0.32 ms) within a single inference account for only 1–3% of the per-token inference latency, demonstrating that these operations impose a negligible impact on overall runtime performance. Regarding memory consumption, our measurements indicate that InternVL-2B (float16) occupies 4.49 GB, accounting for 81.9% of the total memory usage on the end device, and thus representing the primary source of resource utilization. Moreover, applying standard model quantization can further reduce the footprint to below 2.5 GB. Notably, the memory footprint of the auxiliary components remains relatively modest and can be further reduced through optimization strategies, including the adoption of efficient parameter storage techniques. Collectively, these findings demonstrate that the proposed system can be efficiently deployed on conventional end-side hardware without causing significant latency or memory bottlenecks, ensuring both responsiveness and practicality in real-world applications.

5 Related Work

5.1 Efficient Inference for VLM

Recent advancements in Vision-Language Models aim to reduce computational overhead while maintaining performance. LoRA is a leading Parameter-Efficient Fine-Tuning method. Building on this, QLoRA [4] quantizes pre-trained models to 4-bit for reduced memory. VoRA [33] integrates vision-specific LoRA layers into LLMs for visual capabilities. Other LoRA variants, including LoRAM (memory-efficient training) [38] and LoRAT (visual tracking) [21], continue to enhance efficiency. Beyond LoRA, other methods improve VLM inference. FastVLM [32] uses hybrid architectures for efficient high-resolution vision encoding, balancing accuracy and latency. Eve (Efficient Multimodal Vision Language Models with Elastic Visual Experts) [27] employs adaptable visual experts for smaller models. EfficientVLM [35] uses knowledge distillation and modal-adaptive pruning to create fast and accurate VLMs. VisionThink [37] optimizes visual token reduction via reinforcement learning. Moreover, due to the resource limitations of end-devices, these approaches find it difficult to balance efficiency and accuracy on models with a small number of parameters.

5.2 End-Cloud Collaborative Inference Systems

End-cloud collaborative inference aims to leverage complementary resources by placing part of the computation on the end and

the remainder on the cloud. Existing systems mainly follow two paradigms. The first paradigm partitions a *single* model across the end and cloud by selecting cut layers and pipelining execution. Early work such as DeepThings [41] demonstrated dynamic DNN partitioning, and later systems such as DECC [43] and COACH [7] improve pipeline efficiency and mitigate communication bottlenecks. In the VLM/MLLM context, split execution can be combined with representation compression (e.g., AlignedVQ in our evaluation). However, partitioning-based approaches still require transferring intermediate activations across the cloud–end boundary on the critical path, making latency sensitive to RTT even when bandwidth is reduced via compression.

The second paradigm coordinates *multiple* models across the end and cloud, typically pairing a small model for fast drafts with a large cloud model for verification or refinement. Speculative decoding and its multimodal variants [12, 16, 34] exemplify such token-wise collaboration, but they incur frequent cloud-end interactions and amplify RTT overhead. More broadly, hybrid small/large model systems also rely on repeated cloud involvement for difficult cases. In contrast, our approach performs request-wise execution shifting and avoids fine-grained cross-device interaction during generation, while proactively staging task-specific adapters on the end to reduce cold starts and further improve end-side coverage.

6 Conclusion

We presented ShiftVL, a task-aware cloud-end offloading framework for vision-language model serving. ShiftVL serves frequent requests on the end with small VLM augmented by ViTexLoRA, and uses a cloud-hosted large VLM as fallback for complex or long-tail requests, balancing accuracy, latency, and cloud cost. Under tight end-side budgets, ShiftVL uses a predictive adapter scheduler that combines LRU-style caching with imitation learning to proactively stage task-specific adapters. Experiments with InternVL models show that ShiftVL reduces cloud cost by up to 76.3% and end-to-end latency by up to 42.9% while maintaining high multi-task accuracy, demonstrating its practicality for cloud-end VLM deployment.

Acknowledgments

This study is supported by the National Natural Science Foundation of China under Grant Number 62402050.

References

- [1] CHEN, J., LIANG, T., SIU, S., WANG, Z., WANG, K., WANG, Y., NI, Y., ZHU, W., JIANG, Z., LYU, B., ET AL. Mega-bench: Scaling multimodal evaluation to over 500 real-world tasks. *arXiv preprint arXiv:2410.10563* (2024).
- [2] CHEN, Z., WU, J., WANG, W., SU, W., CHEN, G., XING, S., ZHONG, M., ZHANG, Q., ZHU, X., LU, L., ET AL. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2024), pp. 24185–24198.
- [3] CHEN, Z., WU, J., WANG, W., SU, W., CHEN, G., XING, S., ZHONG, M., ZHANG, Q., ZHU, X., LU, L., LI, B., LUO, P., LU, T., QIAO, Y., AND DAI, J. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks, 2024.
- [4] DETTMERS, T., PAGNONI, A., HOLTZMAN, A., AND ZETTMELMOYER, L. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.
- [5] DONG, Z., HE, Q., CHEN, F., JIN, H., GU, T., AND YANG, Y. Edgemove: Pipelining device-edge model training for mobile intelligence. In *Proceedings of the ACM Web Conference 2023* (New York, NY, USA, 2023), WWW '23, Association for Computing Machinery, p. 3142–3153.
- [6] ERDIL, E. Inference economics of language models. *arXiv preprint arXiv:2506.04645* (2025).
- [7] GAO, L., LIU, J., XU, H., XU, S., MA, Q., AND HUANG, L. Accelerating end-cloud collaborative inference via near bubble-free pipeline optimization, 2024.
- [8] GIGANT, T. Design choices for vision language models in 2024. Hugging Face Blog, 2024. [Online]. Available: <https://huggingface.co/blog/vision-language-models>.
- [9] GU, A., AND DAO, T. Mamba: Linear-time sequence modeling with selective state spaces, 2024.
- [10] GUERRERO, P. R., PAN, Y., AND KASHYAP, S. Efficient deployment of vision-language models on mobile devices: A case study on oneplus 13r. *arXiv preprint arXiv:2507.08505* (2025).
- [11] HU, E. J., WALLIS, P., ALLEN-ZHU, Z., LI, Y., WANG, S., WANG, L., CHEN, W., ET AL. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations* (2022).
- [12] HUANG, H., YANG, F., LIU, Z., YIN, X., LI, D., REN, P., AND BARSOUM, E. Specvlm: Fast speculative decoding in vision-language models. *arXiv preprint arXiv:2509.11815* (2025).
- [13] HUANG, H., YANG, F., LIU, Z., YIN, X., LI, D., REN, P., AND BARSOUM, E. Specvlm: Fast speculative decoding in vision-language models. *arXiv preprint arXiv:2509.11815* (2025).
- [14] JIANG, B., HAO, Z., CHO, Y.-M., LI, B., YUAN, Y., CHEN, S., UNGAR, L., TAYLOR, C. J., AND ROTH, D. Know me, respond to me: Benchmarking llms for dynamic user profiling and personalized responses at scale. *arXiv preprint arXiv:2504.14225* (2025).
- [15] JIN, H., AND WU, Y. Ce-collm: Efficient and adaptive large language models through cloud-edge collaboration. In *2025 IEEE International Conference on Web Services (ICWS)* (2025), IEEE, pp. 316–323.
- [16] KANG, J., SHU, H., LI, W., ZHAI, Y., AND CHEN, X. Vispec: Accelerating vision-language models with vision-aware speculative decoding. *arXiv preprint arXiv:2509.15235* (2025).
- [17] KATARE, D., ZHOU, M., CHEN, Y., JANSSEN, M., AND DING, A. Y. Energy-aware vision model partitioning for edge ai. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing* (2025), pp. 671–678.
- [18] LI, B., WANG, R., WANG, G., GE, Y., GE, Y., AND SHAN, Y. Seed-bench: Benchmarking multimodal llms with generative comprehension, 2023.
- [19] LI, J., LI, D., SAVARESE, S., AND HOI, S. Blip-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning* (2023), ICML'23, JMLR.org.
- [20] LI, Y., GUMASTE, D., TURKCAN, M. K., GHADERI, J., ZUSSMAN, G., AND KOSTIC, Z. Distributed vlms: Efficient vision-language processing through cloud-edge collaboration. In *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)* (2025), IEEE Computer Society, pp. 280–286.
- [21] LIN, L., FAN, H., ZHANG, Z., WANG, Y., XU, Y., AND LING, H. Tracking meets lora: Faster training, larger model, stronger performance, 2024.
- [22] LIU, H., LI, C., WU, Q., AND LEE, Y. J. Visual instruction tuning, 2023.
- [23] LIU, X., ZHANG, L., GANESAN, D., AND GUAN, H. Aligned vector quantization for edge-cloud collaborative vision-language models. *arXiv preprint arXiv:2411.05961* (2024).
- [24] LIU, Y., DUAN, H., ZHANG, Y., LI, B., ZHANG, S., ZHAO, W., YUAN, Y., WANG, J., HE, C., LIU, Z., CHEN, K., AND LIN, D. Mmbench: Is your multi-modal model an all-around player?, 2024.
- [25] MAO, Y., HUANG, K., GUAN, C., BAO, G., MO, F., AND XU, J. Dora: Enhancing parameter-efficient fine-tuning with dynamic rank distribution. *arXiv preprint arXiv:2405.17357* (2024).
- [26] MARAFIOTI, A., ZOHAR, O., FARRÉ, M., NOYAN, M., BAKOUCHE, E., CUENCA, P., ZAKKA, C., ALLAL, L. B., LOZHKOVA, A., TAZI, N., ET AL. Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299* (2025).
- [27] RANG, M., BI, Z., LIU, C., TANG, Y., HAN, K., AND WANG, Y. Eve: Efficient multimodal vision language models with elastic visual experts, 2025.
- [28] RESEARCH NESTER. Multimodal AI Market Size, Share & Trends Forecast 2035. [Online]. Available: <https://www.researchnester.com/reports/multimodal-ai-market/6472>, 2025. Accessed: 2025-10-07.
- [29] SHARSHAR, A., KHAN, L. U., ULLAH, W., AND GUIZANI, M. Vision-language models for edge networks: A comprehensive survey. *IEEE Internet of Things Journal* (2025).
- [30] TIAN, C., SHI, Z., GUO, Z., LI, L., AND XU, C. Hydralora: An asymmetric lora architecture for efficient fine-tuning, 2024.
- [31] VAN ROY, B. A short proof of optimality for the min cache replacement algorithm. *Information processing letters* 102, 2-3 (2007), 72–73.
- [32] VASU, P. K. A., FAGHRI, F., LI, C.-L., KOC, C., TRUE, N., ANTONY, A., SANTHANAM, G., GABRIEL, J., GRASCH, P., TUZEL, O., AND POURANSARI, H. Fastvlm: Efficient vision encoding for vision language models, 2025.
- [33] WANG, H., YE, Y., LI, B., NIE, Y., LU, J., TANG, J., WANG, Y., AND HUANG, C. Vision as lora, 2025.
- [34] WANG, S., YU, R., YUAN, Z., YU, C., GAO, F., WANG, Y., AND WONG, D. F. Spec-vla: speculative decoding for vision-language-action models with relaxed acceptance. *arXiv preprint arXiv:2507.22424* (2025).
- [35] WANG, T., ZHOU, W., ZENG, Y., AND ZHANG, X. Efficientvlm: Fast and accurate vision-language models via knowledge distillation and modal-adaptive pruning, 2022.
- [36] XU, J., LI, Z., CHEN, W., WANG, Q., GAO, X., CAI, Q., AND LING, Z. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088* (2024).
- [37] YANG, S., LI, J., LAI, X., YU, B., ZHAO, H., AND JIA, J. Visionthink: Smart and efficient vision language model via reinforcement learning, 2025.
- [38] ZHANG, J., WANG, J., LI, H., SHOU, L., CHEN, K., YOU, Y., XIE, G., GONG, X., AND ZHOU, K. Train small, infer large: Memory-efficient lora training for large language models, 2025.
- [39] ZHANG, Q., CHEN, M., BUKHARIN, A., KARAMPATZAKIS, N., HE, P., CHENG, Y., CHEN, W., AND ZHAO, T. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning, 2023.
- [40] ZHANG, Y.-F., ZHANG, H., TIAN, H., FU, C., ZHANG, S., WU, J., LI, F., WANG, K., WEN, Q., ZHANG, Z., ET AL. Mme-realworld: Could your multimodal llm challenge high-resolution real-world scenarios that are difficult for humans? *arXiv preprint arXiv:2408.13257* (2024).
- [41] ZHAO, Z., BARIJOUGH, K. M., AND GERSTLAUER, A. Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2348–2359.
- [42] ZHU, J., WANG, W., CHEN, Z., LIU, Z., YE, S., GU, L., TIAN, H., DUAN, Y., SU, W., SHAO, J., GAO, Z., CUI, E., WANG, X., CAO, Y., LIU, Y., WEI, X., ZHANG, H., WANG, H., XU, W., LI, H., WANG, J., DENG, N., LI, S., HE, Y., JIANG, T., LUO, J., WANG, Y., HE, C., SHI, B., ZHANG, X., SHAO, W., HE, J., XIONG, Y., QU, W., SUN, P., JIAO, P., LV, H., WU, L., ZHANG, K., DENG, H., GE, J., CHEN, K., WANG, L., DOU, M., LU, L., ZHU, X., LU, T., LIN, D., QIAO, Y., DAI, J., AND WANG, W. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models, 2025.
- [43] ZHUANG, Z., CHEN, J., XU, W., QI, Q., GUO, S., WANG, J., LU, L., YANG, H., AND LIAO, J. DECC: Delay-Aware Edge-Cloud Collaboration for Accelerating DNN Inference. *IEEE Transactions on Emerging Topics in Computing* 13, 02 (Apr. 2025), 438–450.

A Experimental Details

A.1 Hyperparameter Settings

Unless otherwise specified, we use a fixed sliding window of length $H = 100$ to construct user access sequences. The cache size is set to $w = 10$, and the LRU-managed protection region size is set to $v = 3$. The temporal extractor consists of a single Mamba block. The hidden dimension is set to $d = 128$ for both access-side and cache-side embeddings. For ViTextLoRA, we set the rank hyperparameter to $r = 8$ across all datasets and all modality. We use the same evaluation protocol as in the main paper and report averaged results over multiple runs when applicable.

A.2 Implementation Platform

We implemented the end and cloud prototypes of ShiftVL on NVIDIA RTX 2060 Super and A100 GPUs, respectively, connected via a wireless network. We deploy InternVL-2B as the SVLM on the end side

and InternVL-9B as the LVM on the cloud side. Performance evaluation is conducted using simulated user datasets derived from publicly available benchmarks, following the main paper.

A.3 Memory Footprint

Figure 8 reports the memory usage breakdown of major modules during ShiftVL inference, where the length of each part is proportional to its memory usage. This breakdown is useful for understanding which components dominate the end-side footprint.

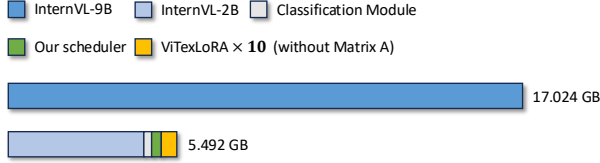


Figure 8: Memory usage breakdown of each module for ShiftVL inference.

B Additional Experimental Results

B.1 Results for Cache Hit Rate (with STD)

Table 3 supplements the corresponding hit-rate figure in the main paper by providing the exact numerical values under different cache sizes, including the standard deviation across runs. We compare ShiftVL with an oracle policy (Belady) and classical cache baselines (LRU/FIFO). We also include an imitation-learning-only variant (IL-only) to isolate the effect of the full design from the learned scheduler alone. Overall, the results show that ShiftVL consistently improves hit rate over classical policies across cache sizes, while remaining below the oracle upper bound as expected.

Table 3: Hit rate comparison under different cache sizes (mean $\pm$ std).

Method	5	10	15	20	25	30
Belady	0.46	0.59	0.67	0.72	0.76	0.79
LRU	0.20	0.34	0.44	0.52	0.57	0.61
FIFO	0.18	0.30	0.39	0.45	0.50	0.55
Ours	0.27 ± 0.047	0.47 ± 0.036	0.55 ± 0.014	0.59 ± 0.010	0.61 ± 0.034	0.62 ± 0.041
IL-only	0.27 ± 0.036	0.43 ± 0.040	0.50 ± 0.041	0.55 ± 0.033	0.56 ± 0.048	0.60 ± 0.038

B.2 Attention Map Analysis of LoRA Variants

To better understand how different parameter-efficient tuning methods affect visual-text alignment, we visualize attention maps for the base model and several LoRA variants. Figures 9 illustrate the qualitative differences in attention allocation across layers. We observe that the most salient changes are concentrated in intermediate-to-deeper transformer layers (approximately layers 16–22): compared with the base model, LoRA-style adaptations tend to re-distribute attention mass toward task-relevant regions, while maintaining similar low-level attention patterns in earlier layers. This layer

range aligns with the common role of mid/deep layers in semantic composition and cross-modal grounding.

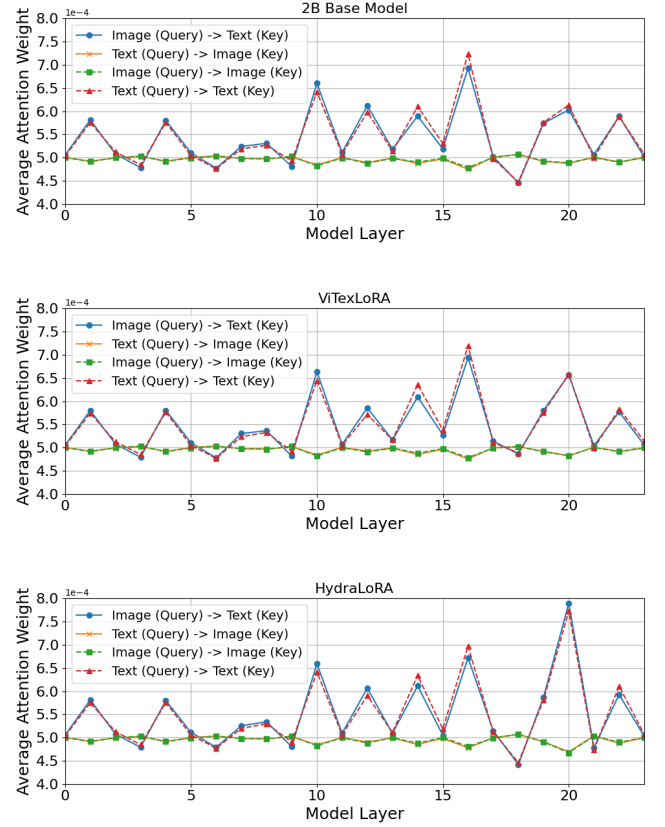


Figure 9: Attention weights per layer visualization for the base model VitextLoRA and HydraLoRA.

QUESTION: Which mood does this image convey?

OPTIONS: A: Melancholic B: Anxious C: **Happy** D: Angry

Pre-Finetune:

1. Visual Analysis:
The image depicts a group of people silhouetted against a large, curved structure, possibly a tunnel or a water slide. The background shows a body of water and a distant shoreline, suggesting an outdoor setting. The sky appears overcast, adding a slightly somber tone to the scene.

2. Question Analysis & Option Evaluation:

- Option A: Melancholic
The image does not convey a melancholic mood. The silhouettes and the overcast sky contribute to a more dynamic and energetic atmosphere rather than a somber one.
- Option B: Anxious
The image does not strongly convey an anxious mood. While the people appear to be in motion, the overall scene is not particularly dramatic or tense.
- Option C: Happy
The image does not strongly convey a happy mood. The silhouettes are in various poses, suggesting movement and excitement, but the overall tone is more energetic than joyful.
- Option D: Angry
The image does not strongly convey an angry mood. The silhouettes are in various poses, but there is no clear indication of anger or aggression.

3. Final Answer:
B. Anxious (though less likely, the image does not strongly convey a specific mood other than dynamic and energetic).

Post-Finetune:

1. Visual Analysis:
The image depicts a group of people silhouetted against a bright sky, seemingly jumping or leaping through a large, circular opening. The background shows a body of water and a distant shoreline, suggesting an outdoor setting. The overall mood appears dynamic and energetic, with the silhouettes creating a sense of movement and action.

2. Question Analysis & Option Evaluation:

- Option A: Melancholic
The image does not convey a melancholic mood. The bright sky, the energetic silhouettes, and the joyful actions of the people suggest a positive and uplifting atmosphere.
- Option B: Anxious
The image does not convey an anxious mood. The sense of freedom and joy in the silhouettes contrasts with the idea of anxiety, which typically involves feelings of worry, fear, or unease.
- Option C: Happy
The image conveys a happy mood. The energetic silhouettes, the bright sky, and the joyful actions of the people all contribute to a positive and uplifting atmosphere.
- Option D: Angry
The image does not convey an angry mood. The sense of freedom and joy in the silhouettes contrasts with the idea of anger, which typically involves feelings of frustration, irritation, or hostility.

3. Final Answer:
C. Happy

Figure 10: Case study: qualitative comparison of InternVL-2B before and after fine-tuning.